CO-1 AT:1 S. Sharvesh

Multiple Users and Processes 192525458

a) Type of OS: A Multi-user, Time-Sharing operating system is suitable for a university Linux server. It allows multiple students to execute programs simultaneously by allocating CPU time slices to different process. Linux supports multitasking, resource sharing, process isolation, and multiple users accounts.

b) Two kernal responsibilities:

1. Process scheduling: Selects processes for CPU execution and performs context switching
2. Memory management: Allocates memory to processes and protects each processe's address space from unauthorized access by other processes

c) C Program using fork():

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
    pid_t pid = fork();
    if (pid < 0){
        perror("fork");
    }       return 1;
```

```
if (pid == 0) {
    printf ("Child PID: %ld \n", (long) getpid());
    printf ("Parent PID: %ld \n", (long getppid());
} else {
    printf ("Parent PID : %ld \n", (long) getpid());
    printf ("Child PID : %ld \n", (long) pid);
    wait(NULL);
}
return 0;
}
```

fork() creates a child process, getpid() returns the current PID, getppid() returns the parent PID and wait() allows the parent to wait for the child

---

Q2. File creation and writing

a) System calls: open(), create(), write(), read() and close() are used to create, access and close files

b) Two system calls

- open() opens a file and returns a file descriptor. O_CREAT creates a missing file, while O_WRONLY opens it for writing
- write() transfers data from memory to a file using its file descriptor

c) C Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd = open("student.txt", O_WRONLY | O_CREAT |
                                  O_TRUNC, 0644);

    char name[] = "Bala\n";
    if (fd == -1) {
        perror("open");
        return 1;
    }
    if (write(fd, name, strlen(name)) == -1) {
    perror("write");
    close(fd);
    return 1;
    }
    close(fd);
    printf("student name saved successfully.\n");
    return 0;
}
```

The program creates student.txt, writes the student's name, and close the file. The permission mode 0644 allows the owner to read and write and others to read, subject to the system's umask

Q3. Copying a file

a) System calls: open(), read(), write() and close()

b) Two system calls:

- read() reads bytes from a source file into a memory buffer. it returns 0 at the end of file and -1 on error
- write() writes bytes from a buffer to the destination file

c) Program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int src, dest;
    char buf[1024];
    ssize_t n, w;
    src = open("source.txt", O_RDONLY);
    if (src == -1) {
        perror("source");
        return 1;
    }
    dest = open("destination.txt", O_WRONLY|O_CREAT
                |O_TRUNC, 0644);
    if (dest == -1) {
        perror("destination");
```

```
    close(src);
    return 1;
  }
while((n=read(src, buf, sizeof(buf)))>0){
  ssize_t off=0;
  while(off<n){
    w=write(dest, buf+off, n-off);
    if(w<=0){
      perror("write");
      close(src);
      close(dest);
      return 1;
    }
    off+=w;
  }
}
if(n==-1)
  perror("read");
  close(src);
  close(dest);
  if(n==-1)
    return 1;
printf("file copied successfully.\n");
return 0;
}
```

The program opens source files, reads data in blocks, writes each block to the destination and closes both files.

The destination contains the copied contents.

Q4. Process Creation and Execution

a) fork() and exec():

- fork() creates a child process
- exec() replaces the current process with a new program.

b) wait() : It makes the parent wait until the child terminates and collects its exit status

c) C Program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
    int pid = fork();
    if (pid == 0){
        execlp("ls", "ls", NULL);
    } else if (pid > 0){
        wait(NULL);
        printf("Child completed \n");
    }
    return 0;
}
```

fork() creates the child, execlp() executes the ls command and wait() makes the parent wait

File Management

a) Importance of file Management

file management organize files, stores and retrieves data, manages storage space, control file permissions and protect sinformation from unauthorized access

b) open() and close():

.open() opens a file and returns a file descriptor.

.close() releases the file descriptor after use

a) C program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd = open("student.text", O_RDONLY);
    char buff[100];
    int n;
    if (fd == -1) {
        perror("open");
        return;
    }
```

```
while ((n = read (fd, buf, sizeof (buf))) > 0)
        write (1, buf, n);
close (fd);
return 0;
}
```

open() opens the file, read() reads the contents, write() displays them on the screen and close() closes the file